A_

// AUDITNET · NETWORK CONFIGURATION AUDITING

Installation and development guide

Architecture, setup, testing, Azure deployment and how to extend the platform.

Version 1.0 · October 2026
English

Contents

Introduction

Architecture

- Components
- Key decisions

Repository and conventions

- Layers of each service
- Conventions

Local installation

- Requirements
- 1. Environment variables
- 2. Databases and support services
- 3. Services
- 4. Console
- 5. Demo data (optional)

Testing

- Service tests
- Screen tests
- In CI

Extending the platform

- A new endpoint
- A migration
- An audit rule type
- A screen
- A translation
- An email
- The guides

Continuous integration and deployment on Azure

- GitHub Actions
- Azure resources
- Create everything from scratch
- Deploy a version by hand
- Deployment security

Operation

- Demo mode and costs

Backups
Periodic review
Email
Diagnosis
Demo data on Azure
Real problems and how they were solved
Permissions by profile and where they are enforced
Where it is enforced

Introduction

AuditNet is a B2B web platform to audit the configurations of Cisco IOS/IOS-XE network devices. It compares the real configuration of each device (running-config) with an expected configuration (baseline), detects deviations, prioritizes them by severity and by device criticality, generates alerts and reports, and keeps the evidence of every audit. It is multi-organization: each client sees only its own data.

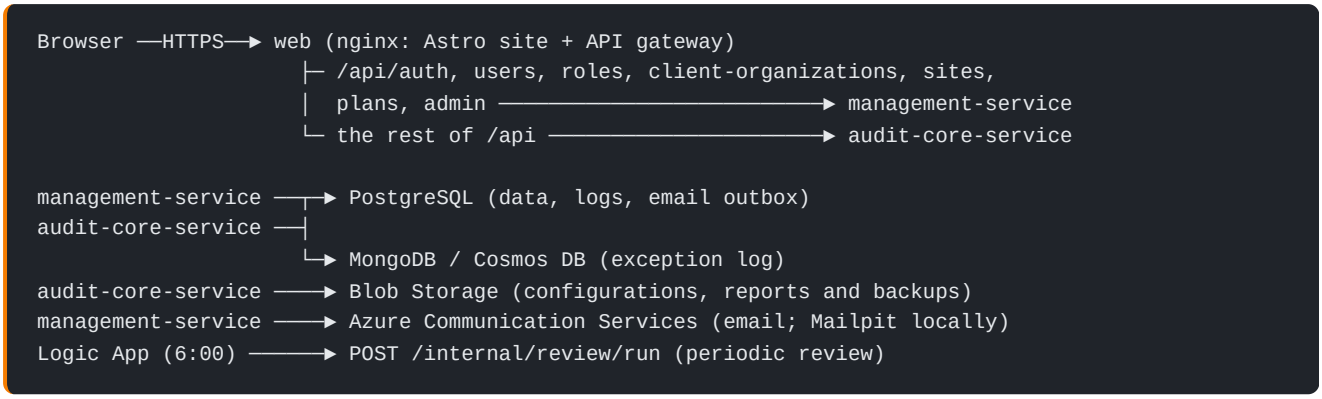
This guide is for whoever has to **install, operate or modify** the platform: developers joining the project and whoever deploys it on Azure. It covers:

- the architecture and why it is built this way;
- how the repository is organized and which conventions it follows;
- how to run it on a local computer and load demo data;
- how to run the tests;
- how to add an endpoint, a migration, a screen, a translation or an email;
- the integration and deployment pipeline on Azure, and day-to-day operation;
- the real problems that came up during development and how they were solved;
- what each profile can do and where it is enforced in the code.

ITEM	VALUE
Repository	`github.com/Leonel-Caruso/tfi-auditnet`
Branches	`main` (current version, deployed to Azure) and `test` (testing, not deployed); `entrega-2` is kept as history
Design decisions	`docs/entrega-2/REGISTRO-DECISIONES.md` (D-01 onwards)
Migrations	`docs/database/MIGRACIONES.md`
Deployment	`docs/deploy/AZURE.md`

> In the commands, whatever goes between `<` and `>` is replaced by the real value (without the signs). Passwords and secrets are never written in the repository or on the command line: the scripts ask for them when they run.

Architecture



› Components

COMPONENT	TECHNOLOGY	RESPONSIBILITY
web	nginx + Astro 5 (static site)	Serves the console and routes `/api/...` to each service on the same origin: the services are not exposed and CORS is not needed. Publishes the content security policy.
management-service (port 8081)	Java 21, Quarkus 3	Sign-in and JWT, users and profiles, organizations and sites, plans, email, system log.
audit-core-service (port 8082)	Java 21, Quarkus 3	Devices, configurations, baselines and rules, audit engine, findings, alerts, dashboard, reports, periodic review, files and backups.
PostgreSQL 17	Flyway per service	One shared database; each service migrates its tables with its own history table.
MongoDB / Cosmos DB	MongoDB API 4.2	Exception log (the NoSQL entity).
Blob Storage / Azurite	private container	Files with their SHA-256.

› Key decisions

- **Two services and one database.** They are split by responsibility (access and administration on one side, auditing on the other) and share PostgreSQL so that scope queries (organization, plan) stay simple. audit-core reads the plan with a native query; that is why management migrates first.
- **Security.** Sign-in returns a signed JWT (HS256) with the user, their organization and their profiles. Each endpoint declares the allowed profiles with `@RolesAllowed` and each service enforces the organization scope. Changing the profile, the organization or the password invalidates open sessions.
- **Three logs.** System (access and security) and transactions (previous and new value of each business operation) in PostgreSQL; exceptions in MongoDB with a configurable level. A tracking code (`X-Request-ID` , set by nginx) links the same operation across the three.

- **Tamper-proof evidence.** Configurations, audits, finding tracking and reports are not modified: there are insert-only triggers and reports are a snapshot.
- **Email with an outbox.** Notices are saved in `correos_salientes` within the same transaction as the event that triggers them and are dispatched afterwards, with retries. This way a provider failure does not lose notices.
- **Multi-language.** The console translates in the browser with dictionaries; error messages are translated with a catalog according to `Accept-Language`; emails use the recipient's language.

Repository and conventions

```
backend/management-service/   Quarkus: access, users, organizations, plans, email
backend/audit-core-service/   Quarkus: inventory, baselines, auditing, findings, reports
frontend/web-app/             Astro: pages, scripts, dictionaries, styles and nginx
docker/                       PostgreSQL, MongoDB, Azurite and Mailpit for local development
infra/azure/                  Azure creation and operation scripts (01 to 09)
scripts/                      load-env.ps1 and datos-demo/ (demo data load)
tests/pantallas/              Screen tests (Playwright and mock API)
tests/datos_demo/             Demo data tests
tests/carga/                  Load test with k6
docs/                         Decisions, migrations, deployment and guides
.github/workflows/            CI, images and deployment
```

› Layers of each service

```
ar/edu/uai/tfi/<service>/
  api/rest/         JAX-RS resources and DTOs: input validation, @RolesAllowed and scope
  application/      use cases and ports (application/port), no infrastructure
  domain/           model (records and enums) and repositories as interfaces
  infrastructure/   adapters: Panache, security, email, logs, files, i18n
```

Dependencies point inwards: `api → application → domain`, and `infrastructure` implements the domain and application ports. That is why the application services are tested with in-memory implementations (`src/test/.../support`).

› Conventions

- **Spanish names** in the domain, the services and the texts; API routes in English (`/api/devices`, `/api/findings`).
- **Simple, defensible code:** no long comments; only those that explain a why.
- **Flyway migrations** in `src/main/resources/db/migration` of each service: `V<n>__<description>.sql`. A migration that has already been applied is never modified; changes go in a new one. History tables: `flyway_historial_management` and `flyway_historial_audit_core`.
- **Decisions** in `REGISTRO-DECISIONES.md` with a `D-xx` number: what was decided, why, alternatives and test.
- **Interface texts** are never written in the HTML or the scripts: always from the dictionaries (see *Extending the platform*).
- **No secrets in Git:** `.env` is ignored; `.env.example` lists the variables without values.

Local installation

› Requirements

TOOL	VERSION
Docker Desktop	with Docker Compose
Java (JDK)	21
Maven	3.9 or later
Node.js	24
Python	3.11 or later (screen tests, demo data and guides)
PowerShell	5 or later (Windows); on Linux or macOS the same steps with `bash`

› 1. Environment variables

Copy `.env.example` to `.env` and fill in the local values: database, `JWT_SECRET`, the initial administrator (`BOOTSTRAP_ADMIN_*`) and the initial organization (`BOOTSTRAP_ORG_*`). `.env` is not pushed to Git.

› 2. Databases and support services

```
.\scripts\load-env.ps1
docker compose --env-file .env -f docker/compose.database.yaml up -d
docker ps
```

It starts `tfi-postgres` (5432), `tfi-mongo` (27017), `tfi-azurite` (10000, storage) and `tfi-mailpit` (emails are read at `http://localhost:8025`).

› 3. Services

In a terminal, **management-service first** (it applies its migrations and creates the profiles, the initial organization and the initial administrator):

```
.\scripts\load-env.ps1
cd .\backend\management-service
mvn quarkus:dev
```

When `Listening on: http://localhost:8081` appears, in another terminal:

```
.\scripts\load-env.ps1
cd .\backend\audit-core-service
mvn quarkus:dev
```

› 4. Console

```
cd .\frontend\web-app
npm install
npm run dev -- --host localhost
```

Open `http://localhost:4321` and sign in with the administrator from `.env`. In development the browser calls the services on ports 8081 and 8082 (it does not go through nginx).

› 5. Demo data (optional)

`scripts/datos-demo` loads 6 fictional organizations with sites, users, devices, baselines, rules and about 100 audits, using the API as an administrator would:

```
python scripts\datos-demo\cargar.py --local --admin <admin_user> --correo
<mailbox>@gmail.com
docker cp .\scripts\datos-demo\historia.sql tfi-postgres:/tmp/historia.sql
docker exec tfi-postgres sh -c 'psql -U $POSTGRES_USER -d $POSTGRES_DB -f
/tmp/historia.sql'
python scripts\datos-demo\cargar.py --local --admin <admin_user> --correo
<mailbox>@gmail.com --reportes
```

- `cargar.py` asks for the administrator's password and the one for the new users; it does not store them. User emails are aliases of the mailbox (`mailbox+user@domain`) and their audit notices are turned off.
- `historia.sql` spreads the newly loaded audits, with their findings and alerts, over the last 8 weeks.
- `--reportes` generates the reports once the dates are spread.
- It can be run again: whatever already exists is reused.

Testing

WHAT	COMMAND	WHERE
Unit and integration tests of each service	`mvn test`	in `backend/<service>`
Frontend types and dictionaries	`npm run check`	in `frontend/web-app`
Screen tests	`npm run build:pruebas` and `python -m pytest tests\pantallas`	build in `frontend/web-app`, pytest from the root
Demo data	`python -m pytest tests\datos_demo`	root
Load (k6)	see `docs/entrega-2/CARGA.md`	root

› Service tests

Application services are tested without a database, with in-memory repositories (`support/EnMemoria...`) and a fixed clock. Each decision in the register states which test backs it. Before pushing changes, both services must pass the full `mvn test`.

› Screen tests

`tests/pantallas` opens the compiled site in Chromium (Playwright) against a mock API (`servidor.py`, data in `datos.py`) and goes through every screen with the five profiles: that they load without errors, that they do not show technical texts, that they fit on a 390 px phone, that they are translated into English and Portuguese and that the dark theme leaves no light backgrounds.

```
cd .\frontend\web-app
npm run build:pruebas
cd ..\..
python -m pip install -r tests\pantallas\requirements.txt
python -m playwright install chromium
python -m pytest tests\pantallas
```

- `build:pruebas` compiles the site to call the API on the same origin, as behind nginx.
- The test server sends the same content security policy as nginx: a script written inside the HTML makes the test fail. That is why waiting for a condition uses `pagina.esperar("<expression>")` and not `wait_for_function`.
- For a new screen: add the route in `test_pantallas.py` and, if it calls new endpoints, their response in `RUTAS` of `servidor.py`.

› In CI

Every push to `main` or `test` runs, on GitHub Actions, the tests of both services, the frontend check and build, the screen tests and the demo data tests. If anything fails, no images are published.

Extending the platform

› A new endpoint

1. **Domain:** if needed, a record in `domain/model` and the methods in the `domain/repository` interface.
2. **Application:** the use case in a service of `application/<module>`. It validates, applies the organization scope and records the operation in the transaction log (`TrazabilidadPort`, with the previous and new values).
3. **Infrastructure:** the repository implementation with Panache in `infrastructure/persistence`.
4. **API:** the resource in `api/rest` with `@Path`, the DTOs as records in `api/rest/dto` and `@RolesAllowed` with the profiles that may use it.
5. **Permissions:** if it is a new action, add it to `MatrizPermisos` (management) and to `RolServiceTest`; the matrix is shown on the Users screen.
6. **Gateway:** if the route prefix is new and belongs to management-service, add it to the expression of `location ~ ^/api/(auth|users|...)` in `frontend/web-app/nginx/default.conf.template`. Everything else goes to audit-core.
7. **Messages:** every new error message the user sees goes in the service's `src/main/resources/i18n/mensajes.txt`, with its `es:`, `en:` and `pt:` lines (if one is missing, it is shown in Spanish).
8. **Tests:** service tests with the in-memory repositories and, if a screen uses it, the mock response in `tests/pantallas/servidor.py`.

› A migration

1. Create `V<n>__<description>.sql` in `db/migration` of the service that owns the table, with the number following the last one of **that** service.
2. Adjust the Panache entity and the repository.
3. Add the row in `docs/database/MIGRACIONES.md`.
4. On startup (`mvn quarkus:dev`, or the Azure deployment) Flyway applies it on its own. Never edit one that has already been applied.

› An audit rule type

The types are `DEBE_CONTENER`, `NO_DEBE_CONTENER` and `VALOR_ESPERADO`. To add one:

1. The value in `TipoReglaConfiguracion` and its evaluation in `MotorAuditoriaService` (it returns whether it passes and the evidence).
2. A migration that widens the `CHECK` of `reglas_baseline.tipo`.
3. In the console, the option in the Rules form, its name in the dictionary (`condicion.<TYPE>`) and, if the evidence has a new sentence, its translation in `lib/evidencia.ts`.
4. Engine tests with configurations that pass and that do not.

› A screen

1. `src/pages/<route>.astro` with `MasterLayout` (the `i18n` attribute sets the key prefix of the header) and its script in `src/scripts/<route>.ts`, loaded with `<script src="../scripts/<route>.ts">`. No scripts are written inside the HTML: the security policy does not allow them.
2. The menu item in `components/Sidebar.astro` (short code and profiles) and the route in the list of `components/Topbar.astro`; optionally words for *Go to module* in `layouts/MasterLayout.astro`.
3. The `menu.<route>` and `ruta.<route>` keys in `i18n/es.ts`, `en.ts` and `pt.ts`, and a dictionary `i18n/pantallas/<screen>.ts` with `es`, `en` and `pt`, added in `i18n/index.ts`.
4. The route in `tests/pantallas/test_pantallas.py` (`PANTALLAS` or `PANTALLAS_POR_PERFIL`).

› A translation

- In the HTML: `data-i18n="key"` for the text and `data-i18n-placeholder`, `data-i18n-title`, `data-i18n-aria-label` or `data-i18n-label` for attributes.
- In the scripts: `t('key', { value })`, `plural(n, 'key')` (keys `key.uno` and `key.otros`) and `etiqueta('group', code)` for codes such as severities or statuses.
- The `Textos<typeof es>` type forces English and Portuguese to have the same keys: `npm run check` fails if one is missing. Data loaded by users (rule names, sites) is not translated.

› An email

The texts are in `management-service/src/main/resources/correo/textos_<language>.properties` and they are assembled in `PlantillasCorreo`. A new type needs its value in `TipoCorreo`, its texts in the three languages and a test in `PlantillasCorreoTest`. It is queued with `CorreoService.encolar(...)` (or, from audit-core, in the shared outbox) and goes out with the usual retries.

› The guides

The sources are in `docs/manuales/<language>/usuario` and `docs/manuales/<language>/desarrollo`, in Markdown. The screenshots are taken automatically from the test site with fictional data (`capturas.py`, `datos_manual.py`):

```
cd .\frontend\web-app; npm run build:pruebas; cd ..\..
python -m pip install -r docs\manuales\requirements.txt
python docs\manuales\generar.py
```

The user guides end up in `frontend/web-app/public/manuales` (the console offers them for download) and this one in `docs/manuales/pdf`.

Continuous integration and deployment on Azure

› GitHub Actions

`.github/workflows/imagenes.yml` runs on every push to `main` or `test` that touches code, tests or the workflow:

JOB	WHAT IT DOES
Tests	`mvn test` of both services
Frontend	`npm ci`, `npm run check` and build
Screen tests	Playwright against the compiled site, and the demo data tests
Images	Publishes the 3 images to GitHub Container Registry with the `sha-<commit>` and branch tags
Deploy to Azure	Only from `main`: waits for the **approval** of the `azure` environment, signs in to Azure with OIDC (no passwords), updates the 3 Container Apps to `sha-<commit>` and runs a smoke test

To approve: *Actions → the run → Review deployments → azure → Approve and deploy*.

› Azure resources

RESOURCE	NAME	PURPOSE
Resource group	`rg-auditnet` (Brazil South)	Everything for the platform
PostgreSQL Flexible Server	`psql-auditnet-lcaruso`, database `tfi_auditnet`	Data and logs (B1ms, 7-day backups)
Container Apps	`ca-auditnet-web`, `ca-auditnet-management`, `ca-auditnet-core` in `cae-auditnet`	The console with public ingress and the two services with internal ingress
Cosmos DB for MongoDB	`cosmos-auditnet-lcaruso`	Exception log (free tier)
Storage	`stauditnetlcaruso`, container `auditnet-archivos`	File repository
Communication Services	Azure managed domain	Email (5 per minute and 10 per hour)
Logic App	daily review at 6:00	Periodic review

› Create everything from scratch

In **Azure Cloud Shell (Bash)**, with the repository cloned:

```
git clone -b main https://github.com/Leonel-Caruso/tfi-auditnet.git
cd tfi-auditnet/infra/azure
bash 01-crear-base.sh            # group, PostgreSQL, database and Container Apps
environment
bash 02-crear-apps.sh            # the 3 Container Apps from the published images
bash 04-crear-cosmos.sh          # exception log
bash 05-crear-storage.sh         # file repository
bash 06-configurar-github-oidc.sh  # deployment from GitHub without passwords
bash 08-crear-email.sh           # email
bash 09-crear-revision.sh        # periodic review
```

The scripts ask for the passwords when they run and store them as Container Apps *secrets*; there are no secrets in the repository. They can be run again. `variables.sh` has the resource names and registers the required providers.

› Deploy a version by hand

```
cd ~/tfi-auditnet/infra/azure && bash 03-actualizar-imagenes.sh sha-<commit>
```

Each service applies its migrations on startup (management first). The administration panel shows the deployed tag.

› Deployment security

- HTTPS required with HSTS; nginx adds `Content-Security-Policy`, `Permissions-Policy`, `X-Frame-Options` and `Referrer-Policy`.
- The services have no public ingress: they can only be reached through nginx. `/internal` is not published, except the periodic review, which requires a 256-bit secret.
- PostgreSQL accepts connections from Azure services (including Cloud Shell). To connect from a PC, a temporary firewall rule with its IP is needed.
- GitHub Actions can only update the 3 apps, and every deployment requires manual approval.

Operation

› Demo mode and costs

Container Apps scale to zero when there is no traffic: the first request takes a few seconds to wake them up. PostgreSQL can be left stopped to save credit. Before a presentation:

```
cd ~/tfi-auditnet/infra/azure
bash 07-modo-demo.sh on       # 1 minimum replica and PostgreSQL started
bash 07-modo-demo.sh estado
bash 07-modo-demo.sh off      # when done: scales to zero again
```

`on` mode consumes quota even without traffic. Costs are monitored in the portal's *Cost Management*.

› Backups

From the **Administration panel → Create backup**: a ZIP with the data of all tables, the schema version and the restore script, stored in the file repository. Azure also keeps automatic PostgreSQL backups for 7 days. How to restore: `docs/database/RESPALDOS.md`.

› Periodic review

A Logic App calls `POST /internal/review/run` every day at 6:00 (Argentina time). For each active organization: it re-audits the devices whose last audit is past the deadline, notifies overdue findings and devices without a new configuration, and the first review of each month sends the summary of the previous month. It can be started by hand from the administration panel. If PostgreSQL was stopped, that day's run fails and the next one catches up.

› Email

Notices go through the outbox (`correos_salientes`); the administration panel shows pending, sent and failed emails, and **Send pending** forces the sending. The Azure managed domain allows few emails per hour: with many new users in a row, emails go out gradually. Locally they are read in Mailpit (`http://localhost:8025`).

› Diagnosis

1. **Administration panel:** status of each service and its components.
2. **Logs:** find the tracking code of the operation and filter the three logs with it.
3. **App logs:** `az containerapp logs show -g rg-auditnet -n ca-auditnet-management --tail 100` (or `ca-auditnet-core`, `ca-auditnet-web`).

› Demo data on Azure

From Cloud Shell (the directory is not kept between sessions: clone or update the repository every time):

```
cd ~ && git clone -b main https://github.com/Leonel-Caruso/tfi-auditnet.git; cd ~/tfi-
auditnet && git pull
SITIO=https://$(az containerapp show -g rg-auditnet -n ca-auditnet-web --query
properties.configuration.ingress.fqdn -o tsv)
python3 scripts/datos-demo/cargar.py --url $SITIO --admin <admin_user> --correo
<mailbox>@gmail.com
export PGPASSWORD=$(az containerapp secret show -g rg-auditnet -n ca-auditnet-management --
secret-name db-password --query value -o tsv)
psql "host=psql-auditnet-lcaruso.postgres.database.azure.com dbname=tfi_auditnet
user=auditnetadmin sslmode=require" -f scripts/datos-demo/historia.sql
unset PGPASSWORD
python3 scripts/datos-demo/cargar.py --url $SITIO --admin <admin_user> --correo
<mailbox>@gmail.com --reportes
```

Real problems and how they were solved

SYMPTOM	CAUSE	SOLUTION
`No plugin found for prefix 'quarkus'`	`mvn quarkus:dev` run from the repository root	Run it inside `backend/<service>`.
audit-core does not start the first time	Its migrations reference management tables	Start management-service first.
The MongoDB driver rejects Cosmos DB	Azure created the account with API 3.6 and the driver requires 4.2	Upgrade it from the portal (*Upgrade MongoDB server version*); `04-crear-cosmos.sh` checks it.
`SubscriptionNotFound` when creating a resource	The provider for that resource type was not registered in the subscription	`variables.sh` registers them first (`registrar_proveedores`).
`ExpressEnvironmentResourceNotSupported`	The Container Apps environment was created in *express* mode, which does not support scheduled jobs	The periodic review is scheduled with a Logic App (`09-crear-revision.sh`).
The smoke test gets `HTTP 000` and the web app has no address	In the Container Apps express environment, updating the image left the web app without ingress	`03-actualizar-imagenes.sh` checks the ingress of each app and enables it again. By hand: `az containerapp ingress enable -n ca-auditnet-web -g rg-auditnet --type external --target-port 8080`.
The first screen takes several seconds	The apps scaled to zero or PostgreSQL is stopped	`07-modo-demo.sh on` before presenting.
Cloud Shell does not find the repository or the variables	The Cloud Shell session is ephemeral	Clone or `git pull` and define the variables again in every session.

SYMPTOM	CAUSE	SOLUTION
`unrecognized arguments` in `az postgres flexible-server firewall-rule`	The CLI changed its syntax	Use `--server-name <server> --name <rule>`. From Cloud Shell no rule is needed: it gets in through the Azure services rule.
Welcome emails take a while to arrive	The Azure managed domain is limited to 10 emails per hour	Wait; the outbox retries them. For more volume, a verified custom domain.
A site script does not run on Azure but does in development	The security policy blocks scripts written inside the HTML	Scripts in separate files; Astro configured with `assetsInlineLimit: 0`.
`wait_for_function` fails in the screen tests	It evaluates text as code and the policy blocks it	Use `pagina.esperar(...)`.
Playwright does not find the browser	The installed version does not match the package	`python -m playwright install chromium`.
The account is locked while testing a script	5 wrong passwords in a row	Wait 15 minutes or unlock it from Users.
After deploying, the previous version shows	The browser cached the page	Reload with Ctrl+F5 (files in `/_astro/` are hashed; the HTML is always revalidated).
Your session ended without signing out	The profile, organization or password changed, or the token expired	Sign in again: this is the expected behavior.

Permissions by profile and where they are enforced

MODULE · ACTION	ADM	ORG	ANL	AUD	RSP
Dashboard, client status, devices, baselines, rules, history, findings, alerts and reports · view	✔	✔	✔	✔	✔
Devices · register, edit and disable	✔	✔			
Device types · register	✔				
Baselines and rules · create, version and edit	✔			✔	
Configurations · import; audits · run	✔		✔	✔	
Findings · change the tracking status	✔		✔	✔	
Critical alerts · assign or return	✔		✔		✔
Reports · generate	✔		✔	✔	
Organizations · register, edit and disable	✔				
Sites · register, edit and disable	✔	✔			
Users · manage administrators	✔				
Users · manage operational profiles	✔	✔			
Plan · view the requests	✔	✔			✔
Plan · request a change		✔			
Plans · assign, approve requests and edit prices	✔				
Logs and administration panel	✔				

ADM: system administrator · ORG: organization administrator · ANL: network analyst · AUD: technical auditor · RSP: IT management lead.

› Where it is enforced

LAYER	WHERE	WHAT IT CONTROLS
API	`@RolesAllowed` on each resource in `api/rest`	Which profiles can call each endpoint (403 otherwise).
Scope	`UsuarioAutenticado` (audit-core), `Actor` and the user and site services (management)	That every profile, except the system administrator, works only on its own organization.
Plan	`ControlPlan` and `PlanService`	Device and user limits and the read-only mode of an expired Piloto.

LAYER	WHERE	WHAT IT CONTROLS
Documentation	`MatrizPermisos` and `RolServiceTest`	The table shown on the Users screen; it must be updated if a permission changes.
Console	`Sidebar.astro` (profiles of each item) and the `data-roles` / `data-admin-only` attributes	Which menus and buttons are shown. It is only a convenience: security is enforced by the API.
Tests	`test_perfiles.py`, `test_pantallas.py` and each service's tests	That each profile sees and can do what it should.